

JUNGLE JUGGLE

1ST MILESTONE

Team:

- Kymyz Crew

Members:

- Adil Zhetigenov - RHZ142
- Muslim Abdyvapov - A6PYQA
- Nicolas Nino Lozano - A0T4ZR

- Feasibility Plan: Assess the practicality of the project.

We plan to develop Safari, a 2D simulation game, using Unity, a powerful and flexible game engine that supports complex video games behavior, real-time simulations, and UI management. Our team consists of three members, each responsible for different aspects of the project:

Unity provides built-in physics, AI pathfinding, and state management tools, which will allow us to implement:

- Dynamic animal behavior, including movement, hunger/thirst mechanics, reproduction, and group migration.
- Environmental interactions, such as herbivores consuming plants and carnivores hunting prey.
- Economy and management system, allowing players to purchase resources and earn revenue.
- Tourist simulation, including jeeps following roads and affecting player revenue.

We will use GitLab for version control and Trello for task management, ensuring smooth collaboration and tracking of progress.

The game will be developed over the semester according to the given dates, following an agile workflow with weekly check-ins will keep the project on track and prevent mistakes.

Functional Requirements

Game Mechanics

- The game must feature a 2D top-down map.
- Players must be able to switch between at least three speed levels (hour/day/week) at any time.
- The game must support at least three levels of difficulty.

Environment

- The map must contain bushes, trees, and grassy areas.
- Players can purchase and place plants on the map.
- Small water sources must be available by default, and players can construct ponds.

Animals

- The safari must include both carnivorous and herbivorous animals.
- Herbivores must be able to eat trees, bushes, and grass.
- Carnivores must hunt and feed on herbivores.
- Animals must need water to survive.
- Animals must age over time and consume more resources as they grow older.
- Animals must form and migrate in groups.
- Groups containing adults must have the ability to reproduce.
- Well-fed animals must rest before selecting a new destination.
- Hungry or thirsty animals must actively search for food and water sources they have discovered.

Tourism & Jeeps

- Tourists must be able to rent jeeps for safaris.
- Each jeep must have a capacity of up to four passengers.
- Players must be able to purchase jeeps.

Roads

- The safari must have an entrance and an exit.
- Players must build navigable roads connecting the entrance and exit.
- Vehicles must randomly choose a path to transport tourists and return empty.
- Capital Management
- Players must start with an initial capital.
- Capital must be used to purchase plants, animals, jeeps, roads, and other tools.
- Revenue must be generated through animal sales and jeep rentals.
- The number of tourists must depend on the diversity and quantity of animals present.

Non-Functional Requirements

Performance

- The game must run smoothly on mid-range hardware with minimal lag.
- Simulations (e.g., animal movement, resource consumption) must not degrade performance.

Usability

- The user interface must be intuitive and easy to navigate.
- The game must provide tooltips or tutorials to help players understand game mechanics.

Scalability

- The game must support a reasonable number of animals, plants, and tourists without performance issues.
- The system must be designed to allow future expansions or updates.

Reliability & Availability

- The game must have an auto-save feature to prevent progress loss.
-
- Crashes or unexpected errors should not corrupt saved data.
-
- AI & Behavior Simulation
- Animals must exhibit realistic behaviors based on their hunger, thirst, and group tendencies.
- Tourist behavior must be varied to enhance realism.

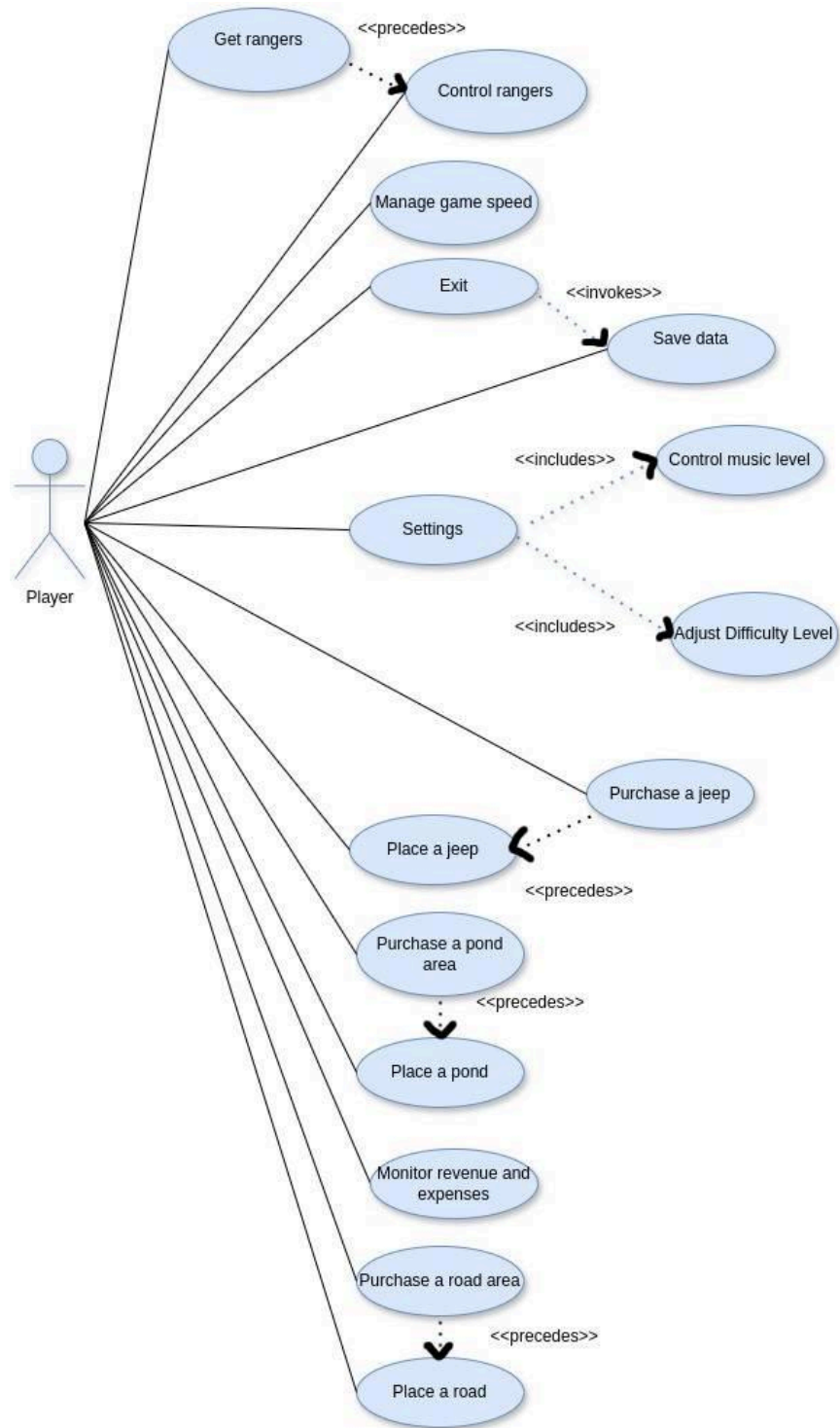
Graphics & Audio

- The game must have visually distinct elements for different plants, animals, and objects.
- Background music and sound effects must enhance the gameplay experience.

Compatibility

- The game must run on at least Windows
- The game must support different screen resolutions.
- Security
- The game must prevent exploits, such as infinite money or unintended behaviors.
- Save files must be protected against tampering.

- **Use-Case Diagram: Create a user-centered diagram showing interactions.**



- **User Stories:** Write detailed user stories for testing and development.

Game Speed and Difficulty

1. As a player, I want to switch between different game speeds (hour/day/week) at any time so that I can control the pacing of the game.
2. As a player, I want to select from at least three difficulty levels so that I can choose a challenge that matches my experience and skill level.

Plants and Water Areas

3. As a player, I want the game world to contain bushes, trees, and grassy areas so that herbivores have natural food sources.
4. As a player, I want to purchase and place plants on the map so that I can provide additional food sources for herbivores.
5. As a player, I want small water sources to exist naturally on the map so that animals have a basic water supply.
6. As a player, I want to build ponds so that I can provide more water sources for animals.

Animals

7. As a player, I want carnivores and herbivores to roam freely within the safari so that I can observe their natural behaviors.
8. As a player, I want herbivores to eat trees, bushes, and grass so that they can survive and grow.
9. As a player, I want carnivores to hunt and eat herbivores so that they can survive.
10. As a player, I want animals to need water so that I must manage water sources effectively.
11. As a player, I want animals to age and consume more resources over time so that I must plan for sustainability.
12. As a player, I want animals to reproduce when groups contain adult individuals so that the ecosystem remains dynamic.
13. As a player, I want hungry or thirsty animals to search for food or water they have already discovered so that I can predict their movements and plan accordingly.

Jeeps and Tourism

14. As a player, I want tourists to rent jeeps to explore the safari so that I can generate revenue.
15. As a player, I want each jeep to carry up to four passengers so that tourism operations are efficient.
16. As a player, I want to purchase jeeps so that I can control the number of tourists visiting the safari.

Roads and Navigation

17. As a player, I want the safari to have an entrance and an exit so that vehicles have a defined path.
18. As a player, I want to build navigable roads that connect the entrance and exit so that vehicles can move properly.
19. As a player, I want jeeps to return empty to the entrance after dropping off tourists so that vehicle flow remains continuous.

20. As a player, I want an initial capital to purchase necessary resources so that I can start managing the safari.
21. As a player, I want to earn revenue from selling animals so that I can sustain and expand my safari.
22. As a player, I want to earn revenue from jeep rentals so that I can make money from tourism.
23. As a player, I want the number of tourists to depend on the diversity and number of animals they can see so that I am incentivized to maintain a thriving ecosystem.

24. As a player, I want to purchase poachers so that they can control the number of certain animal species.

25. As a player, I want to control the poachers to hunt animals that pose a threat to the ecosystem.

26. As a player, I want a warning if my capital is critically low so that I can take action to avoid bankruptcy.

27. As a player, I want an auto-save feature so that I do not lose progress due to an unexpected issue.

```

classDiagram
    class Tree {
        +map: Map
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Grass {
        +map: Map
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Bush {
        +map: Map
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Minimap {
        +map: Map
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Map {
        +get: Location() Vector2
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Player {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Ranger {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Person {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Ranger {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Transport {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Jump {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Animal {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Herbivore {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Carnivore {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class WaterSource {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class Pond {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class GameWindow {
        +map: Map
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class TouristController {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class RangerController {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    class AnimalController {
        +name: String
        +movement: Vector2
        +position: Vector2
        +value: double
        +width: float
        +height: float
    }
    Tree --> Map
    Grass --> Map
    Bush --> Map
    Minimap --> Map
    Map --> Player
    Map --> Ranger
    Map --> Person
    Map --> Ranger
    Map --> Transport
    Map --> Jump
    Map --> Animal
    Map --> Herbivore
    Map --> Carnivore
    Map --> WaterSource
    Map --> Pond
    GameWindow --> TouristController
    GameWindow --> RangerController
    GameWindow --> AnimalController
    TouristController --> RangerController
    RangerController --> AnimalController
    AnimalController --> GameWindow
  
```

The diagram illustrates the MVC pattern for a game. The **MODEL** section contains the game entities: **Tree**, **Grass**, **Bush**, **Minimap**, **Map**, **Player**, **Ranger**, **Person**, **Ranger**, **Transport**, **Jump**, **Animal**, **Herbivore**, **Carnivore**, **WaterSource**, and **Pond**. The **VIEW** section contains the user interface components: **GameWindow**, **TouristController**, **RangerController**, and **AnimalController**. The **CONTROLLER** section contains the logic for handling user input and game state: **GameWindow**, **TouristController**, **RangerController**, and **AnimalController**. The diagram shows the relationships between these classes, including inheritance and associations.

